

Laser Vault Heist

Computer Graphics Fundamentals

Final Project Report

Students: Nurbek Yerbulekov, Makhmet Demesh, Nurassyl Nygmet

Group: SE-2428

Topic 12

C++17 / OpenGL 3.3 Core

Contents

1 Introduction 4

1.1 Aim and objectives 4

1.2 Team contributions 5

2 Literature Review 5

2.1 Illumination and reflectance 5

2.2 Skeletal deformation and animation 6

2.3 Collision and visibility 7

2.4 Display mapping, shadows and textures 7

2.5 Journal context 8

3 Project Description 9

3.1 Gameplay and scope 9

3.2 Environment design 9

3.3 Controls and settings 10

4 Technical Implementation 11

4.1 Application structure 11

4.2 Tools and resources 11

4.3 Transformations and hierarchy 12

4.4 Laser collision and simulation 12

4.5 Cameras and mouse input 13

4.6 Character and animation 13

5 Graphics and Rendering 15

5.1 Render passes and CCTV 15

5.2 Lighting and shading 15

5.3 Textures and display conversion 17

5.4 Alarm and victory sequence 18

6 Testing and Evaluation 20

6.1 Functional regression tests 20

6.2 Performance method 20

6.3 Measured results 21

6.4 Defects and corrections 21

7 Conclusion and Future Work 22

8 References 23

8.1 Technical and asset credits 25

9 Appendix 26

9.1 Technical requirement mapping 26

9.2 Build and evidence files 26

1 Introduction

Laser Vault Heist is a real-time 3D stealth game set inside a mansion vault. The player observes moving laser barriers, chooses when to advance, crouches through restricted gaps and opens a mechanical safe. Contact with a laser triggers an alarm and a retry. Successful traversal ends with an animated safe opening and a character dance.

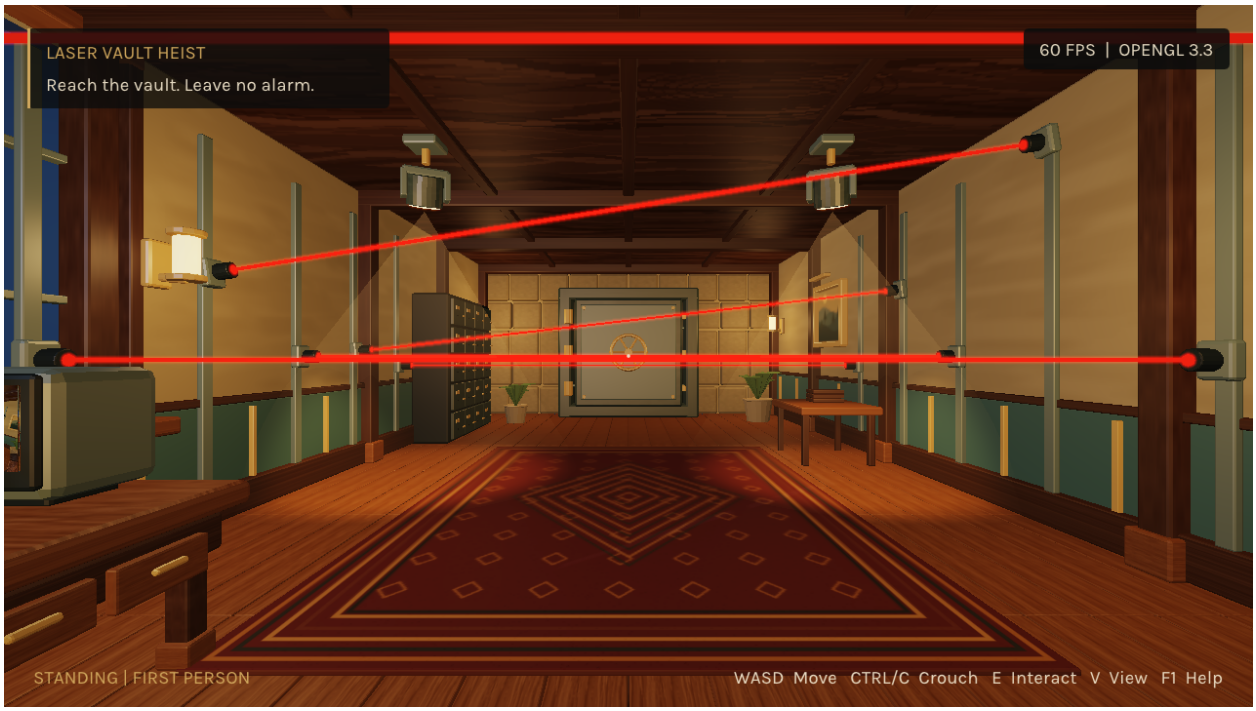


Figure 1. Vault interior with textured furnishings, moving laser barriers and the live CCTV monitor.

The project combines the central components of a modern graphics pipeline: indexed geometry, model/view/projection transformations, hierarchical animation, per-fragment shading, texture sampling and interactive cameras. The application is implemented in C++17 with a custom OpenGL 3.3 Core renderer and separate GLSL programs.

1.1 Aim and objectives

The aim is to turn these graphics techniques into a coherent playable sequence. The main objectives are to keep laser rendering aligned with collision, support both camera perspectives, animate a skinned character, provide visible feedback for interactions, and sustain real-time performance. The environment and safe must provide sufficient clearance for both gameplay and the final door animation.

1.2 Team contributions

Student	Contribution
Nurbek Yerbulekov	Project lead; game design, visual direction, character and environment asset work, animation and visual integration.
Makhmet Demesh	Gameplay mechanics: laser collision, progression and game states; rendering and technical integration.
Nurassyl Nygmet	Gameplay mechanics: controls and interactions; regression tests, builds and documentation.

2 Literature Review

The literature review compares eighteen journal articles across illumination, skeletal deformation, collision, visibility, tone mapping, shadows and textures. The comparison distinguishes methods used in the application from more general alternatives. Complete article metadata and DOI links appear in Section 8.

2.1 Illumination and reflectance

[1] Phong compares shading rules and relates image quality to normals, hidden-surface removal and perception. The project evaluates lighting per fragment. It uses the Blinn half-vector variant rather than copying Phong's original reflection-vector expression.

[2] Cook and Torrance model reflection using surface microgeometry and material-dependent directional response. This is the physical comparison for the vault's brass and painted metal. The implemented empirical Blinn-Phong shader is cheaper and easier to explain, but it does not reproduce a calibrated metal BRDF.

[3] Schlick develops a compact reflectance model that balances physical constraints with practical cost. Its efficiency goal supports keeping material controls understandable. The game does not implement the full Schlick BRDF or claim physically based rendering.

The vault uses per-fragment Blinn-Phong shading because its diffuse, specular and attenuation terms are explicit and inexpensive. More physically detailed reflectance models would improve material fidelity, particularly for metal, but would require a different material representation and additional calibration.

2.2 Skeletal deformation and animation

[4] Kavan and colleagues replace linear matrix blending with approximate dual-quaternion blending to reduce collapsing-joint artifacts. The thief uses linear blend skinning because its weights and motions are simple. This paper identifies a concrete future improvement for shoulder and knee volume preservation.

[5] Kavan, Sloan and O'Sullivan fit standard linear skinning to animated meshes with sparse weights and efficient optimization. The character retains four normalized influences per vertex. Existing skin weights are imported and checked; this optimization method is a comparison rather than an implemented weight-fitting stage.

[6] Le and Deng jointly recover rigid bone transforms and sparse convex skinning weights from example poses. Export checks weight normalization and non-negativity while retaining the supplied skeleton. Automatic recovery of joints and weights is outside the asset pipeline.

[16] Merry, Marais and Gain generalize skeletal subspace deformation while retaining a linear formulation and hardware rendering. The comparison explains why plain linear blend skinning can lose volume around strongly bent joints. Our constrained animation range and garment shapes mitigate, but do not mathematically eliminate, those artifacts.

[17] Mohr and Gleicher fit a compact real-time deformation model to examples exported from richer authoring tools. Editable Blender assets are converted to indexed meshes and sampled local transforms. The project uses imported motion and derived clips rather than fitting a physically simulated character.

[18] James and Twigg derive skinning approximations from deformable mesh sequences using automatically chosen transforms and weights. The character uses 52 weighted bones within a 64-matrix shader palette. Automatic bone discovery and reconstruction from mesh sequences are comparison methods rather than runtime features.

Linear blend skinning fits the character and the available palette size. The exporter retains normalized imported weights and sampled local transforms. It does not solve for a new skeleton or optimize weights. Foot-contact adjustments and a lower crouch pelvis address the specific poses used by the game.

2.3 Collision and visibility

[7] Klosowski and colleagues organize discrete-orientation bounding volumes into hierarchies for efficient collision queries in complex environments. A hierarchy is unnecessary for six laser boxes and a small set of furniture colliders. Transforming body samples into each beam's local space keeps the project inexpensive and directly inspectable.

[8] Jimenez, Thomas and Torras classify continuous and sampled collision strategies and explain how temporal and spatial pruning control cost. The game adopts repeated static local-box tests at 240 Hz, with a conservative body margin. This is bounded sampling, not an exact continuous-collision solver for arbitrary velocities.

[13] Teschner and colleagues survey deformable collision using hierarchies, distance fields, spatial partitioning, image-space and stochastic methods. The animated thief is represented by a sampled upright body for gameplay. This avoids deforming triangle-level collision, while pose-bound checks keep the crouched mesh compatible with the chosen clearance.

[9] Cohen-Or and colleagues survey visibility algorithms for interactive walkthroughs, including techniques that avoid drawing hidden geometry. The compact single room uses depth testing, back-face culling and static instancing. More complex occlusion systems would add overhead without a demonstrated benefit at this scene size.

The hazard set is small: six moving beam volumes and a bounded collection of environment proxies. Inverse-transform point tests at a fixed simulation rate offer a compact solution. Continuous collision and general deformable-body methods would be appropriate for a broader range of speeds and objects.

2.4 Display mapping, shadows and textures

[10] Reinhard and colleagues adapt photographic tone reproduction to map high-dynamic-range images to a display. The project also renders into a floating-point target before display conversion. Its implemented global fitted curve is credited separately and is not Reinhard's photographic operator.

[11] Larson, Rushmeier and Piatko preserve visibility through tone reproduction informed by luminance adaptation and human vision. This provides a criterion for tuning the night scene: the safe and hazards must remain readable. The game uses artistic tuning rather than the paper's histogram and visual-adaptation model.

[12] Hasenfratz and colleagues compare real-time shadow methods by geometry, lighting conditions, image quality and cost. The vault uses one directional depth map with a small PCF kernel. The spotlights illuminate surfaces but do not each render a separate shadow map; their dust cones are an inexpensive visual approximation.

[14] Blinn and Newell use surface parameterization to map patterns and filter their contributions over a pixel footprint. The project's UV coordinates and mipmaps address the same sampling problem. Its mesh rasterizer does not reproduce the original patch renderer or environment-reflection system.

[15] Lefebvre and Hoppe synthesize controllable, deterministic textures through parallel neighborhood matching and multiresolution operations. The game instead generates seeded painted textures offline. This produces reproducible wood, cloth and plaster without the runtime complexity of exemplar-based synthesis.

The rendering pipeline combines an HDR target, a fitted filmic display curve, filtered shadow comparisons and prepared texture maps. These choices improve readability while keeping the implementation small. Laser glow additionally uses scene depth to preserve foreground occlusion.

2.5 Journal context

The source audit records journal categories and public reproductions of SJR 2025 rankings. These rankings describe the named journals in that year; they do not assign historical quartiles to the original article publication years. Sixteen reviewed articles remain when the two Communications of the ACM entries are excluded.

Journal	Recorded category and quartile
ACM Transactions on Graphics	Computer Graphics and Computer-Aided Design; Q1
Computer Graphics Forum	Computer Graphics and Computer-Aided Design; Q1
IEEE Transactions on Visualization and Computer Graphics	Computer Graphics and Computer-Aided Design; Q1
Computers & Graphics	Computer Graphics and Computer-Aided Design; Q2
Communications of the ACM	Computer Science (miscellaneous); Q1

Ranking-source links are retained with the article metadata in docs/references-verified.json and evidence/source-audit.json. The ranking records use accessible reproductions because direct SCImago and Scopus access was unavailable during the audit.

3 Project Description

3.1 Gameplay and scope

The playable route contains six laser barriers with vertical movement and rotation. The player can walk, wait and crouch, toggle a desk lamp, change the live security-camera view and interact with the safe. The safe becomes usable only after crossing the final barrier. The two possible outcomes are an alarm followed by retry, or a complete safe-opening finale followed by victory.

The project includes the three topic extensions: rotating spotlights, live CCTV and safe-opening victory. It contains one player, one vault and one route. Combat, guard characters, multiplayer and additional levels are outside the implemented scope; audio is not included.

3.2 Environment design

The vault is approximately 8 m wide, 15 m long and 4 m high. Wood, plaster, stone and brass establish a consistent interior palette. Furniture, framed art, plants, deposit drawers, security fixtures and the safe organize the room around the central traversal path. The final approach provides at least 1.70 m clearance between the door sweep and the last laser.

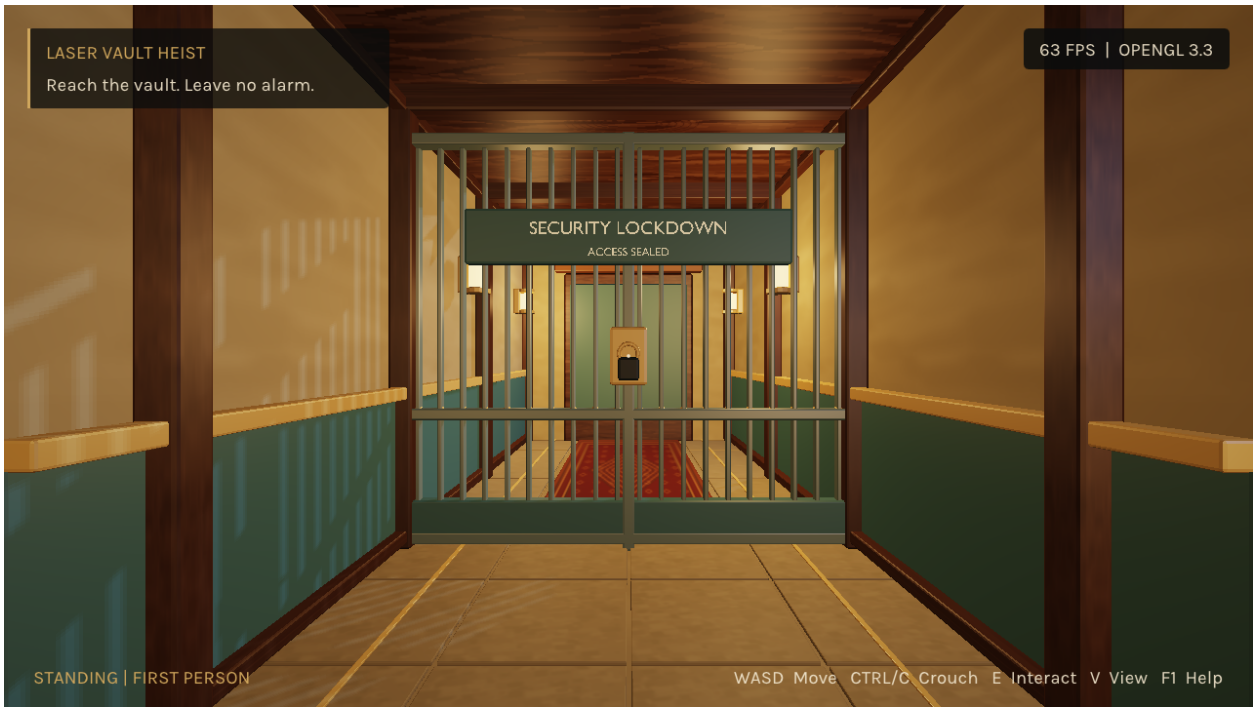


Figure 2. Illuminated entrance corridor behind the locked security gate.

An enclosed vestibule and archive corridor continue the view behind the spawn point. A barred gate, padlock and lockdown sign give a visible reason that the route is blocked. Player and camera limits stop before the gate. Paintings and the deposit

cabinet are separated from emitter mounting rails; the evaluated layout contains no targeted decoration/security overlaps.

3.3 Controls and settings



Figure 3. Main menu with mouse sensitivity and monitor-aware frame-rate settings.

Input	Action
Enter	Begin
WASD / mouse	Move and look around
V	Switch first- and third-person cameras
Hold Ctrl or C	Crouch
E	Use the targeted lamp, CCTV or safe within 2.15 m
Esc / F1	Pause and settings / controls help
R	Retry after alarm or victory
F12	Capture a screenshot

Mouse sensitivity uses a multiplier with 0.022 degrees per count per unit. Auto FPS follows the active display refresh rate, presets include reported monitor modes, Custom accepts 1-10000 FPS, and Unlimited disables pacing. Retry preserves the selected sensitivity, frame limit and camera mode.

4 Technical Implementation

4.1 Application structure

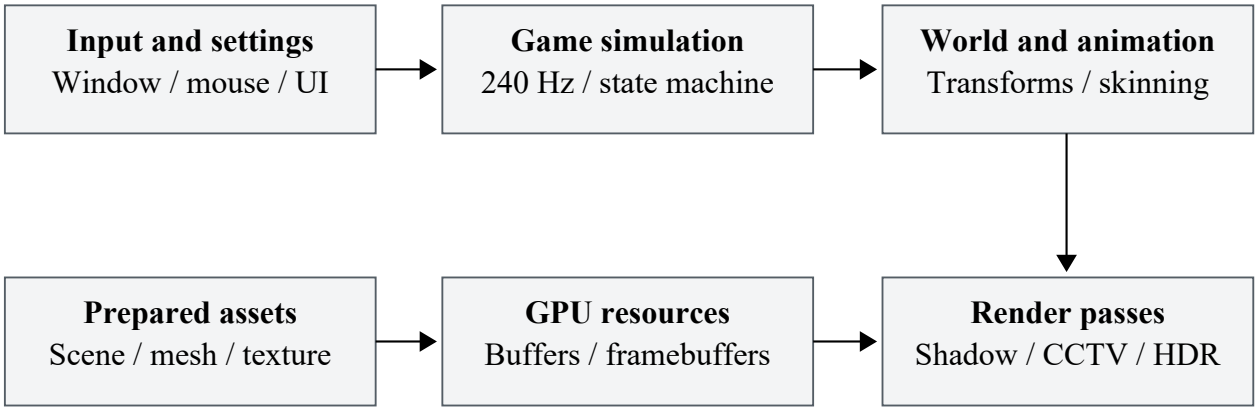


Figure 4. Application data flow from input and assets to simulation, animation and rendering.

The frame loop collects input, advances fixed-step gameplay, evaluates scene and character transforms, submits render passes and draws the interface. Static geometry and textures are uploaded once. Safe components, lasers, spotlight equipment and the character update through dynamic transforms or a bone palette.

Module	Responsibility
main.cpp / mouse_motion.hpp	Window, input collection and application loop
game.cpp / fixed_step.hpp	Player movement, contacts, interactions and state transitions
scene.cpp / graphics.cpp	Hierarchy, mesh and texture loading, buffers and framebuffers
animation.cpp / world.cpp	Clip sampling, skinning palette and animated scene transforms
renderer.cpp / shaders	Shadow, CCTV, HDR scene and display passes
ui.cpp / frame_limiter.hpp	Menus, settings and steady-clock frame pacing

4.2 Tools and resources

The runtime uses C++17 and GLSL 330. GLFW provides the window and context, GLAD loads OpenGL entry points, GLM supplies mathematical types, stb handles image input/output and Dear ImGui draws interface panels. CMake builds the application and C++ tests. Runtime dependencies are included in the repository.

Blender stores editable geometry, the rig and animation actions. Prepared LVHM meshes, scene manifests and LVHA animation samples are loaded by the application. Cameras, lights, collision, animation playback and rendering are evaluated in C++ and GLSL. GLB files are retained as exchange copies. The runtime does not depend on the authoring applications.

4.3 Transformations and hierarchy

Node transforms compose from parent to child. Model, view and projection matrices map a local position into clip space. The normal matrix is the inverse transpose of the model matrix, preserving perpendicularity when scale is non-uniform.

$$M_{world} = M_{parent} \times T \times R \times S$$
$$p_{clip} = P \times V \times M_{world} \times p_{local}$$
$$N_{world} = \text{normalize}(\text{transpose}(\text{inverse}(\text{mat3}(M_{world}))) \times N_{local})$$

The safe door rotates around a hinge node; the wheel and bolts inherit its motion. Spotlight fixtures use parent-child transforms so that their visible equipment and light direction remain aligned. Blender coordinates map once from (x, y, z) to runtime (x, z, -y), with one unit equal to one metre.

4.4 Laser collision and simulation

Rendering and contact tests share each beam model matrix. A body sample in world space is transformed into the beam local frame. Contact occurs when the transformed point lies inside all three padded half-extents. This directly handles a rotated beam without enlarging it to a world-aligned box.

$$q = \text{inverse}(M_{beam}) \times (p_{body}, 1)$$
$$\text{hit when } |q_x| \le h_x \text{ and } |q_y| \le h_y \text{ and } |q_z| \le h_z$$

Body samples span the upright proxy at 0.08 m intervals, including its exact top. Standing height is 1.78 m and crouched height is 1.33 m. Gameplay advances at 240 Hz, movement is normalized diagonally, and one-shot actions remain pending until a simulation step consumes them. Room and furniture proxies constrain movement. The method is discrete body sampling rather than continuous collision detection.

4.5 Cameras and mouse input

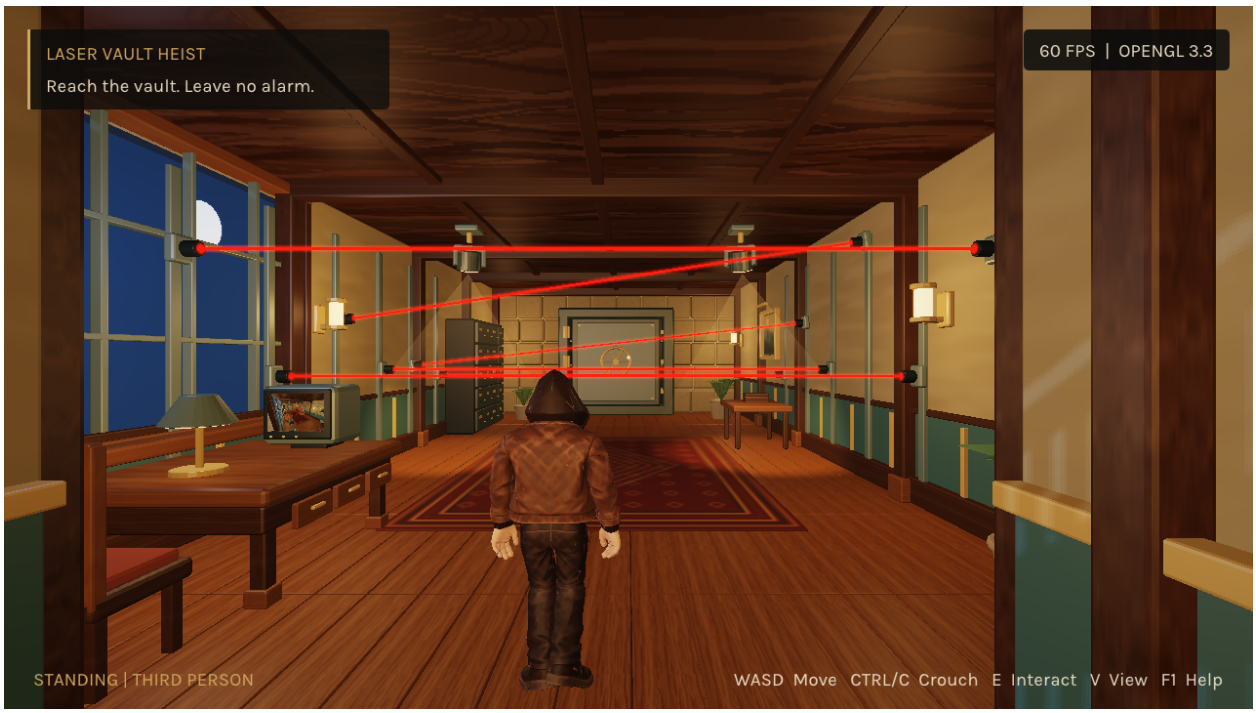


Figure 5. Third-person view with the animated character inside the vault.

First-person view aligns the character with the mouse heading. Third-person view allows independent orbit while the body faces actual standing movement. Directional crouch clips are selected relative to body heading. The third-person boom is swept against padded obstacle proxies and refined near a hit; shortening the boom preserves the look direction. The body is hidden in the main view when the camera is too close to the hood.

Windows mouse movement is collected from WM_INPUT. Relative deltas are accumulated and consumed once per rendered frame; absolute devices maintain separate baselines. Focus and capture changes reset pending motion. Camera rotation uses input counts rather than frame time. Frame pacing changes render frequency without changing the fixed simulation step.

4.6 Character and animation

The character contains 30,725 triangles and 52 weighted bones, within the 64-matrix shader palette. Each vertex retains at most four normalized influences. Animation stores sampled local translation, rotation and scale. Translation and scale interpolate linearly; rotations use quaternion interpolation. Global pose transforms combine with inverse bind matrices before upload.

$$p_{skinned} = \sum w_i \times B_i \times p_{bind}, \text{ for } i = 1...4$$
$$\sum w_i = 1, w_i \geq 0$$
$$B_i = G_{pose,i} \times \text{inverse}(G_{bind,i})$$

Group	Runtime clips
Standing	Idle, Walk, WalkBackward, Run
Crouching	CrouchIdle, CrouchWalk, CrouchBackward, CrouchLeft, CrouchRight
Actions	Interact, Dance, DanceLong

Forward crouching reverses the backward source cycle; backward walking reverses the walking cycle. Resting poses combine opposing gait phases, while the interaction pose adds an arm reach. Crouch preparation lowers the pelvis by 0.14 m and adjusts leg contact. Runtime locomotion phase advances with travelled distance.



Figure 6. Skinned character playing the derived forward-crouch cycle.

Tests evaluate actual skinned vertices over the crouch frames. The maximum checked height is approximately 1.287 m, below the 1.33 m body proxy. Eye heights are 1.57 m standing and 1.22 m crouched, keeping view height consistent with the visible stance.

5 Graphics and Rendering

5.1 Render passes and CCTV

Meshes use VAO/VBO/EBO with separate vertex and fragment shaders. The frame schedules a directional depth pass, a CCTV scene pass, the main HDR scene and display conversion. Static objects share mesh/material instance groups. Framebuffer completeness is checked, and depth testing and back-face culling reject hidden opaque fragments.



Figure 7. Live CCTV framebuffer containing the current room and animated character.

The security camera renders to a 640 × 360 color/depth framebuffer. Its monitor is excluded from that pass to avoid feedback. The physical monitor samples the linear security image during scene rendering; an additional display-converted target supplies the enlarged interface view. The main HDR target uses RGBA16F color and a depth texture, while the directional shadow target is 2048 × 2048.

5.2 Lighting and shading

Blinn-Phong lighting is evaluated per fragment. Eight point lights illuminate the vault and entrance corridor; two rotating spotlights follow their fixture hierarchy. Ambient and directional moon terms establish the night palette. The desk lamp is toggleable and alarm feedback changes security lighting to red.

$$C_{out} = ambient + \Sigma\ visibility \times attenuation \times cone \times C_{light} \times [k_d \times C_{base} \times \max(N \cdot L, 0) + k_s \times \max(N \cdot H, 0)^{s}] + emission$$

$$H = normalize(L + V)$$

$$attenuation = power / (1 + 0.20d + 0.095d^2)$$

N, L and V are normalized surface-normal, light and view vectors. H is the half-vector, s controls specular sharpness and d is distance to the light. Spotlight factors interpolate inner and outer cone limits. Directional shadows use a 3 × 3 percentage-closer filtering kernel. Local lights do not each have a separate shadow map.



Figure 8. Desk-lamp interaction changes the local light and visible lamp glow.



Figure 9. Rotating security fixtures and their spotlight illumination.

The spotlight direction is obtained from the same world transform as its visible mount. Additive cone geometry makes the beam direction legible; it is a visualization of the lighting region rather than physically integrated volumetric scattering.

5.3 Textures and display conversion

Environment surfaces use UV-mapped texture images; the character uses its base-color texture. Images are uploaded once, decoded from sRGB for lighting and assigned generated mipmaps with trilinear minification. The current shader uses Blinn-Phong material parameters; it does not implement a metallic/roughness material model.

A fitted filmic tone curve [19] maps HDR color into display range, followed by gamma encoding and a restrained vignette. Laser glow samples nearby bright pixels. To preserve foreground occlusion, the filter reconstructs linear eye distance from the depth texture and rejects emission samples farther than the current surface by a depth tolerance.

$$\text{zndc} = 2 \times \text{depth} - 1$$
$$\text{deye} = 2 \times \text{near} \times \text{far} / (\text{far} + \text{near} - \text{zndc} \times (\text{far} - \text{near}))$$

A GPU regression scene compares absent, rear and foreground beams around the character. Depth-aware glow removes the measured 114-pixel spill from the rear beam while preserving a foreground beam. The opaque surface and its glow therefore follow compatible visibility rules.

5.4 Alarm and victory sequence



Figure 10. Crouched view below the final laser before the safe approach.

The final passage uses the same body sampling and laser transforms as the rest of the route. Crossing it unlocks safe interaction. Contact at any stage enters Alarm, changes visual feedback and offers retry; retry reconstructs the route state and laser phases.

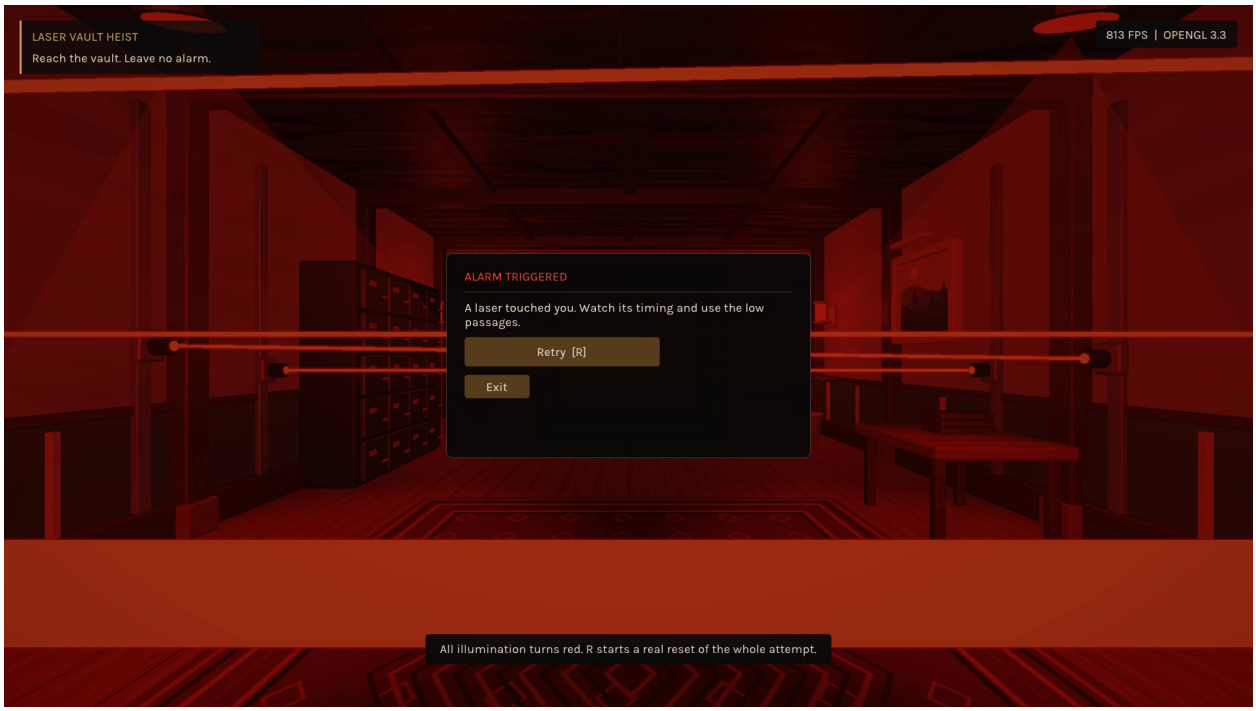


Figure 11. Alarm feedback after laser contact, with retry controls.

Safe interaction starts a 3.4-second mechanical opening: the wheel rotates, bolts retract and the door swings clear. The character then appears against the open safe and plays the 7.5-second dance once. Movement and interactions are locked during the finale; pause freezes it. Victory follows the complete 10.9-second sequence.



Figure 12. One-shot character dance against the fully opened safe.



Figure 13. Victory state after the complete safe-opening and dance sequence.

6 Testing and Evaluation

6.1 Functional regression tests

The Release build passes all seven CTest suites. Logic tests cover bounded inputs and complete state progressions; animation tests inspect loaded runtime data; the rendering regression uses a real OpenGL context. This separates gameplay assertions from image-based visibility checks.

Suite	Coverage
Laser occlusion	Rear/foreground beam visibility, depth-aware glow and 2 cm inner-door bolt clearance
Gameplay mathematics	Oriented contact, crouch clearance, state changes and interactions
Complete route	Movement through all six barriers and safe interaction
Demo and slow frames	State progression, retained one-shot inputs and slow-frame consistency
Animation clips	Weights, clip duration and skinned crouch bounds
Victory cinematic	Full door/dance duration, movement lock, pause, last pose and retry
Camera modes	Orbit, directional movement, input consumption, obstacle limits, settings and third-person completion

Runtime screenshots document the room, cameras, interactions, failure and finale. Automated route controllers supply ordinary movement and interaction inputs to the same gameplay logic. Collision remains active throughout route verification.

6.2 Performance method

The measurement computer uses an Intel Core i5-13400F, NVIDIA GeForce RTX 5060, approximately 32 GiB RAM and Windows x64. The driver reports OpenGL 3.3.0 NVIDIA 617.42. Three visible 1280 × 720 runs each last 25 seconds; the first five seconds are excluded as warm-up. The scene includes live CCTV, moving hazards, skinning and the safe sequence. VSync, frame limiting and recording are disabled.

Frame timing covers the complete loop, including simulation, draw submission, interface, telemetry and presentation. Average FPS is 1000 divided by mean frame time in milliseconds. The 1% low is 1000 divided by the mean of the slowest 1% of frame times; frame-time percentiles use nearest rank.

6.3 Measured results

Run	Avg FPS	1% low	p50 ms	p95 ms	p99 ms
1	1148.9	695.8	0.833	1.147	1.320
2	1185.3	726.5	0.810	1.081	1.265
3	1168.2	660.3	0.818	1.115	1.338

Run	Measured frames	Peak working set	Peak private
1	22979	140.9 MiB	252.2 MiB
2	23705	144.9 MiB	254.1 MiB
3	23364	140.9 MiB	254.2 MiB

Average throughput ranges from 1148.9 to 1185.3 FPS, and 1% lows range from 660.3 to 726.5 FPS. Peak working set remains below 145 MiB in these runs. The results describe this machine and route, with CPU wall-clock timing rather than isolated GPU timestamps. Different hardware, operating-system scheduling and workload can change both averages and tail latency.

Normal play defaults to a cap matched to the active display. The limiter controls frame pacing independently of the 240 Hz simulation and mouse-count interpretation. Uncapped figures measure available throughput rather than the number of distinct images a monitor can display.

6.4 Defects and corrections

Defect	Correction
Uncontrolled view rotation	Consume fresh WM_INPUT deltas once; reset per-device baselines on focus/capture changes.
Laser glow across the character	Reject glow samples behind the visible foreground depth.
Safe-door flicker	Separate retracted bolt faces from the inner door plane by 2 cm.
Intersecting decorations	Move painting frames and deposit furniture clear of emitter rails.
Empty view behind spawn	Add the enclosed corridor, a visible locked gate and matching collision.
Door sweep near the final beam	Extend the safe approach and maintain at least 1.70 m clearance.

7 Conclusion and Future Work

Laser Vault Heist integrates a playable stealth route with a custom C++/OpenGL renderer, two camera modes, skinned character animation, dynamic lighting, live CCTV and an articulated safe. The failure/retry loop and complete dance finale provide a clear beginning, challenge and outcome. The environment supports those mechanics through visible boundaries and consistent hazard clearance.

The principal technical result is agreement between systems: rendering and collision share beam transforms, camera obstruction preserves look direction, input consumes fresh motion once, and post-processing respects foreground depth. Seven passing regression suites and the recorded Windows measurements support the tested implementation.

The current limitations are finite body sampling, linear blend skinning, one directional shadow map, empirical material shading and validation on Windows x64. Future improvements include swept collision for higher speeds, improved shadow stability, deformation quality for extreme poses, additional platform validation and audio/accessibility settings. These extensions can build on the existing module boundaries without changing the central vault route.

8 References

[1] Bui Tuong Phong, "Illumination for computer generated pictures", Communications of the ACM, vol. 18, no. 6, pp. 311-317, 1975. DOI: 10.1145/360825.360839.

https://doi.org/10.1145/360825.360839

[2] R. L. Cook, K. E. Torrance, "A Reflectance Model for Computer Graphics", ACM Transactions on Graphics, vol. 1, no. 1, pp. 7-24, 1982. DOI: 10.1145/357290.357293.

https://doi.org/10.1145/357290.357293

[3] Christophe Schlick, "An Inexpensive BRDF Model for Physically-based Rendering", Computer Graphics Forum, vol. 13, no. 3, pp. 233-246, 1994. DOI: 10.1111/1467-8659.1330233.

https://doi.org/10.1111/1467-8659.1330233

[4] Ladislav Kavan, Steven Collins, Jiří Žára, Carol O'Sullivan, "Geometric skinning with approximate dual quaternion blending", ACM Transactions on Graphics, vol. 27, no. 4, pp. 1-23, 2008. DOI: 10.1145/1409625.1409627.

https://doi.org/10.1145/1409625.1409627

[5] L. Kavan, P.-P. Sloan, C. O'Sullivan, "Fast and Efficient Skinning of Animated Meshes", Computer Graphics Forum, vol. 29, no. 2, pp. 327-336, 2010. DOI: 10.1111/j.1467-8659.2009.01602.x.

https://doi.org/10.1111/j.1467-8659.2009.01602.x

[6] Binh Huy Le, Zhigang Deng, "Smooth skinning decomposition with rigid bones", ACM Transactions on Graphics, vol. 31, no. 6, pp. 1-10, 2012. DOI: 10.1145/2366145.2366218.

https://doi.org/10.1145/2366145.2366218

[7] J.T. Klosowski, M. Held, J.S.B. Mitchell, H. Sowizral, K. Zikan, "Efficient collision detection using bounding volume hierarchies of k-DOPs", IEEE Transactions on Visualization and Computer Graphics, vol. 4, no. 1, pp. 21-36, 1998. DOI: 10.1109/2945.675649.

https://doi.org/10.1109/2945.675649

[8] P. Jiménez, F. Thomas, C. Torras, "3D collision detection: a survey", Computers & Graphics, vol. 25, no. 2, pp. 269-285, 2001. DOI: 10.1016/S0097-8493(00)00130-8.

https://doi.org/10.1016/s0097-8493(00)00130-8

[9] D. Cohen-Or, Y.L. Chrysanthou, C.T. Silva, F. Durand, "A survey of visibility for walkthrough applications", IEEE Transactions on Visualization and Computer Graphics, vol. 9, no. 3, pp. 412-431, 2003. DOI: 10.1109/TVCG.2003.1207447.

https://doi.org/10.1109/tvcg.2003.1207447

[10] Erik Reinhard, Michael Stark, Peter Shirley, James Ferwerda, "Photographic tone reproduction for digital images", ACM Transactions on Graphics, vol. 21, no. 3, pp. 267-276, 2002. DOI: 10.1145/566654.566575.

https://doi.org/10.1145/566654.566575

[11] G.W. Larson, H. Rushmeier, C. Piatko, "A visibility matching tone reproduction operator for high dynamic range scenes", IEEE Transactions on Visualization and Computer Graphics, vol. 3, no. 4, pp. 291-306, 1997. DOI: 10.1109/2945.646233.

https://doi.org/10.1109/2945.646233

[12] J. -M. Hasenfratz, M. Lapierre, N. Holzschuch, F. Sillion, "A Survey of Real-time Soft Shadows Algorithms", Computer Graphics Forum, vol. 22, no. 4, pp. 753-774, 2003. DOI: 10.1111/j.1467-8659.2003.00722.x.

https://doi.org/10.1111/j.1467-8659.2003.00722.x

[13] M. Teschner, S. Kimmerle, B. Heidelberger, G. Zachmann, L. Raghupathi, A. Fuhrmann, M.-P. Cani, F. Faure, N. Magnenat-Thalmann, W. Strasser, P. Volino, "Collision Detection for Deformable Objects", Computer Graphics Forum, vol. 24, no. 1, pp. 61-81, 2005. DOI: 10.1111/j.1467-8659.2005.00829.x.

https://doi.org/10.1111/j.1467-8659.2005.00829.x

[14] James F. Blinn, Martin E. Newell, "Texture and reflection in computer generated images", Communications of the ACM, vol. 19, no. 10, pp. 542-547, 1976. DOI: 10.1145/360349.360353.

https://doi.org/10.1145/360349.360353

[15] Sylvain Lefebvre, Hugues Hoppe, "Parallel controllable texture synthesis", ACM Transactions on Graphics, vol. 24, no. 3, pp. 777-786, 2005. DOI: 10.1145/1073204.1073261.

https://doi.org/10.1145/1073204.1073261

[16] Bruce Merry, Patrick Marais, James Gain, "Animation space: a truly linear framework for character animation", ACM Transactions on Graphics, vol. 25, no. 4, pp. 1400-1423, 2006. DOI: 10.1145/1183287.1183294.

https://doi.org/10.1145/1183287.1183294

[17] Alex Mohr, Michael Gleicher, "Building efficient, accurate character skins from examples", ACM Transactions on Graphics, vol. 22, no. 3, pp. 562-568, 2003. DOI: 10.1145/882262.882308.

https://doi.org/10.1145/882262.882308

[18] Doug L. James, Christopher D. Twigg, "Skinning mesh animations", ACM Transactions on Graphics, vol. 24, no. 3, pp. 399-407, 2005. DOI: 10.1145/1073204.1073206.

https://doi.org/10.1145/1073204.1073206

[19] K. Narkowicz, "ACES Filmic Tone Mapping Curve", 2016. Fitted expression used for HDR display conversion; CC0 license option.

https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/

8.1 Technical and asset credits

Runtime libraries are GLFW (zlib/libpng), GLM (MIT option), Dear ImGui (MIT), stb (MIT option), generated GLAD declarations (CC0 with registry/generator notices), and Karla by Jonathan Pinhorn (SIL OFL 1.1). Exact versions, upstream links and license notices are retained in CREDITS.md, docs/dependency-lock.json and LICENSES/.

The character originates from a Meshy model package, with animation preparation through Mixamo. The original FBX, textures, skeleton and seven source motion actions are preserved. Runtime integration derives the additional crouch-forward, backward-walk, idle and reach clips. Environment geometry and textures were prepared for this project in Blender. Just Another Night Shift served as a style reference; no game assets or code were extracted.

9 Appendix

9.1 Technical requirement mapping

ID	Implementation
R1	OpenGL 3.3 Core, indexed buffers and separate GLSL programs
R2	Vault, corridor, furnishings, fixtures, safe and animated character
R3	Model/view/projection and parent-child transforms
R4	Per-fragment lighting, moving spots and toggleable lamp
R5	UV textures, generated mipmaps and trilinear filtering
R6	Two cameras and lamp/CCTV/safe interactions
R7	Visible FPS counter and measured 1280 × 720 performance
R8	CMake source build with included runtime dependencies

9.2 Build and evidence files

README.md provides the exact Release build and CTest commands. The application runs with assets/ and shaders/ beside the executable. Native resource layouts are documented in docs/runtime-formats.md and module responsibilities in docs/architecture.md.

Figure screenshots are stored in evidence/screenshots/report-current/. The three benchmark CSVs and their summary are in evidence/benchmarks-current/. Regression output is in evidence/ctest-current.txt. The original character sources and editable Blender scenes remain under art/.